Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

Overview

Neural Bayes estimators learn the map from a dataset straight to its posterior estimate — giving fast, *amortised* Bayesian inference.

- **The catch:** training one is *itself* a Monte Carlo problem — it needs many simulated samples, and each gradient step is noisy.
- **This thesis:** treat that as **variance reduction** — cut the noise and train faster, on fewer samples.

Idea 1 · Rao–Blackwellisation

Replace noisy targets by their exact conditional mean. **Provably** lower variance — needs a *conjugate* block.

Idea 2 · Importance sampling

Draw from a proposal and reweight. **General** — but with *no guarantee*.

Background: Neural Bayes estimators

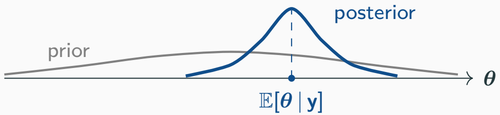
Bayesian inference

Everything we do sits within a **Bayesian inference** framework.

- Fix a model: parameters $\boldsymbol{\theta}$ with prior $p(\boldsymbol{\theta})$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} \mid \boldsymbol{\theta})$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

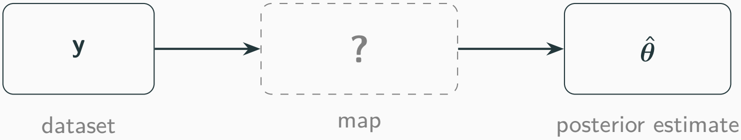
$$p(\boldsymbol{\theta} \mid \mathbf{y}) \;\propto\; p(\mathbf{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

- The posterior is our full, updated belief about $\boldsymbol{\theta}$.



Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ.
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta \mid \mathbf{y}]$ — in principle a **deterministic mapping** from the data $\mathbf{y}$ to $\hat{\theta}$.
- But it is rarely available in closed form — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:



The Bayes risk

Which point estimate $\hat{\boldsymbol{\theta}}(\mathbf{y})$ should we return? Fix a **loss** $\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$ and minimise the **Bayes risk**

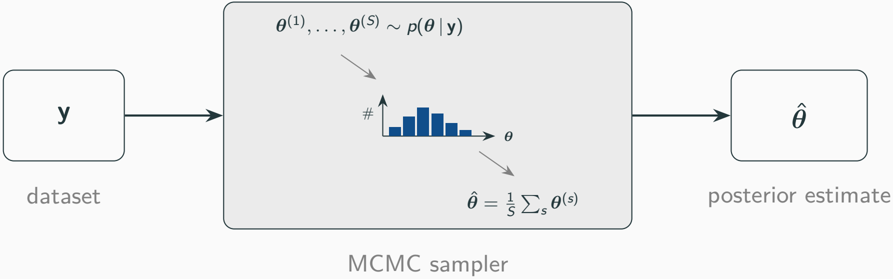
$$\mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\big[\ell\big(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}(\mathbf{y})\big)\big].$$

The choice of loss sets *which* posterior summary is optimal:

loss $\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$	optimal estimate (its minimiser)
squared $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2$	posterior **mean** $\mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$
absolute $\lvert\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\rvert$	posterior **median**
quantile $(\alpha - \mathbf{1}_{\{\boldsymbol{\theta} < \hat{\boldsymbol{\theta}}\}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$	posterior **quantile**

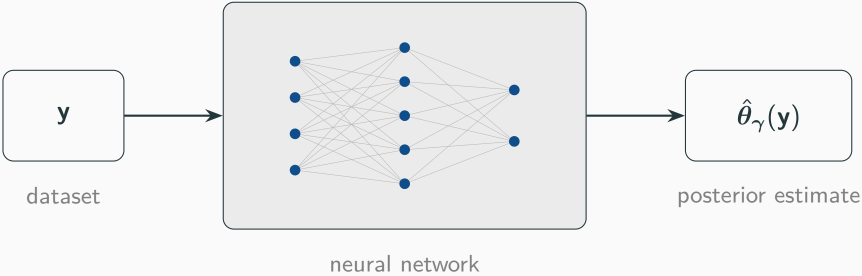
Usually this criterion is used to *choose* which estimator to use.

Method (1): Markov chain Monte Carlo



- $\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(S)}$ come from a **Markov chain** with the posterior as its stationary distribution — accurate and general, but can be slow to converge.
- And it must be re-run from scratch for every new dataset $\mathbf{y}$ — costly when you have many.

Method (2): Neural Bayes estimator



- A **neural network** $\hat{\boldsymbol{\theta}}_\gamma(\cdot)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs $(\boldsymbol{\theta}, \mathbf{y})$ and adjust γ so $\hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}) \approx \boldsymbol{\theta}$.
- Then each new dataset is a single **forward pass** — the cost is amortised.

Training a neural Bayes estimator

Goal: choose the network weights γ that minimise

$$L(\gamma) = \mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\left[\ell\big(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y})\big)\right].$$

No closed form — so each step estimates it by **Monte Carlo** on a minibatch and takes a **stochastic-gradient** step:

$$\widehat{L}(\gamma) = \frac{1}{B}\sum_{b=1}^{B} \ell\big(\boldsymbol{\theta}^{[b]}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}^{[b]})\big), \qquad (\boldsymbol{\theta}^{[b]}, \mathbf{y}^{[b]}) \sim \text{model},$$

$$\gamma^{\text{new}} = \gamma - \eta\, \nabla_\gamma \widehat{L}(\gamma).$$

Insight: the minibatch gradient $\nabla_\gamma \widehat{L}$ is a **random estimate** — it has variance, so we can apply variance-reduction techniques to speed up and stabilise training, and save simulation cost.

Idea 1: Rao–Blackwellisation

The trick

The training target $\boldsymbol{\theta}$ is a **noisy draw**. Split $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$; in the minibatch loss estimator, replace each term by its **conditional mean** given $(\boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]})$ — averaging out the noise in $\boldsymbol{\theta}_1$:

$$\widehat{L}_{\mathrm{MC}}(\gamma) = \frac{1}{B}\sum_{b=1}^{B} \ell(\boldsymbol{\theta}^{[b]}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}^{[b]})),$$

$$\widehat{L}_{\mathrm{RB}}(\gamma) = \frac{1}{B}\sum_{b=1}^{B} \mathbb{E}\left[\, \ell(\boldsymbol{\theta}^{[b]}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}^{[b]})) \,\middle|\, \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]} \,\right].$$

- Same mean — but the **Rao–Blackwell theorem** makes $\widehat{L}_{\mathrm{RB}}$ have never larger variance, for any model and any architecture.
- Exactly computable when the block $\boldsymbol{\theta}_1$ is **conditionally conjugate**.

Squared error: a clean target

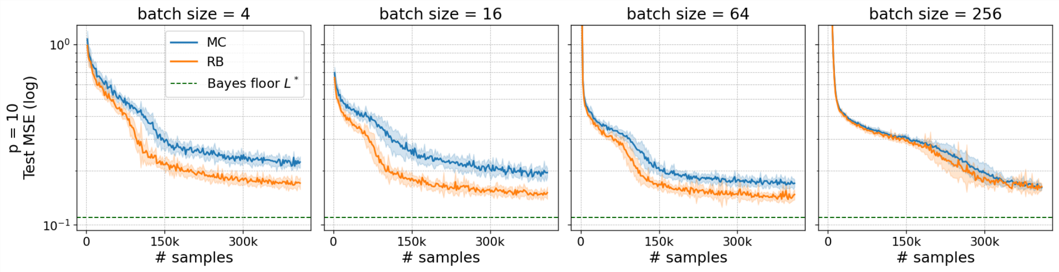
For **squared-error** loss the conditional mean is simple — expand each term of $\widehat{L}_{\mathrm{RB}}$:

$$\widehat{L}_{\mathrm{RB}}(\gamma) = \frac{1}{B}\sum_{b=1}^{B}\Big[\, \big\|\hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}^{[b]}) - \mathbb{E}[\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]}]\big\|^2 + \underbrace{\mathrm{Tr}\big(\,\mathrm{Var}(\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]})\big)}_{\text{constant in } \gamma}\,\Big].$$

- Just squared error to a **clean target** $\mathbb{E}[\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]}]$ — the exact conditional mean, in place of the noisy draw $\boldsymbol{\theta}^{[b]}$.
- The variance term is **constant in** γ: it drops straight out of the gradient.

Bayesian linear regression: $\boldsymbol{\theta} = (\boldsymbol{\beta}, \sigma^2)$ with Gaussian prior $\boldsymbol{\beta} \sim \mathcal{N}(0, \sigma_0^2 I)$; condition on σ^2 and $\boldsymbol{\beta}$ is Gaussian, so the clean target $\mathbb{E}[\boldsymbol{\beta} \mid \sigma^2, \mathbf{y}]$ is **closed-form**.

Does it pay off?



Bayesian linear regression, $p = 10$ coefficients; mean $\pm$ 1 s.d. over repeated training runs.

- The Gibbs sampler gives the Bayes-optimal floor L^* (green).
- RB reaches a **lower test error** at the small and moderate batches — where the training gradient is noisiest.
- The advantage **shrinks as the batch grows**.

Idea 2: Importance sampling

Importance sampling: A more general lever

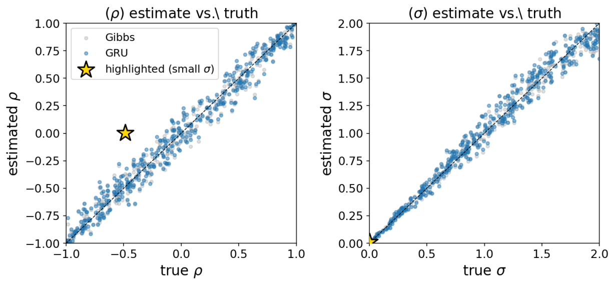
Another commonly used variance-reduction technique is **importance sampling** — and, unlike Rao–Blackwell, it needs **no conjugacy**.

$$\widehat{L}_{\mathrm{IS}}(\gamma) = \frac{1}{B}\sum_{b=1}^{B} w(\boldsymbol{\theta}^{[b]})\,\ell(\boldsymbol{\theta}^{[b]}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}^{[b]})), \qquad \boldsymbol{\theta}^{[b]} \sim q, \quad w(\boldsymbol{\theta}) = \tfrac{p(\boldsymbol{\theta})}{q(\boldsymbol{\theta})}.$$

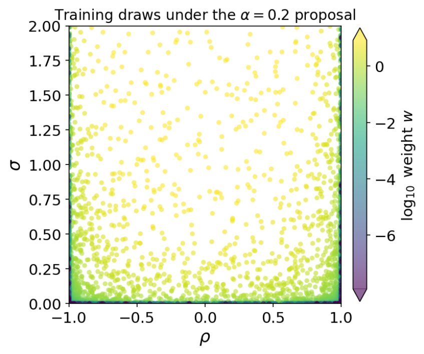
- Draw the parameters from a **proposal** $q(\boldsymbol{\theta})$ instead of the prior $p(\boldsymbol{\theta})$, and **reweight** each draw.
- Unbiased for any proposal q — the weight $w = p/q$ undoes the change of measure.
- In general, a **careful choice of** q can lower the variance.
- But no guarantee — a poor proposal *inflates* the variance instead.

A test case: AR(1)

Oversampling edges in the AR(1) model: $x_t = \rho\, x_{t-1} + \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, priors $\rho \sim \mathcal{U}(-1, 1)$, $\sigma \sim \mathcal{U}(0, A)$.

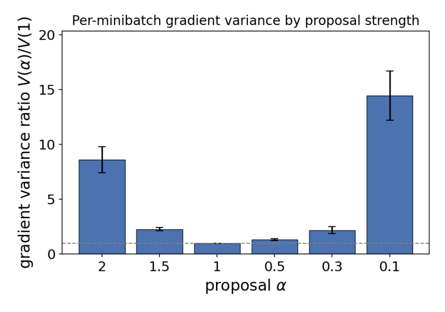


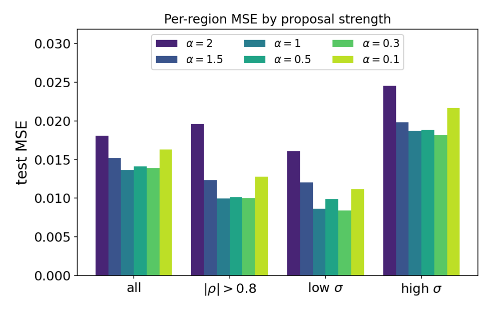
The GRU tracks Gibbs; the star (tiny σ) is **unidentifiable**.



Draws pile on the edges.

Importance sampling: the result





Every proposal **inflates** the training-gradient variance ($V(\alpha)/V(1) \geq 1$) — the very quantity RB shrinks.

And the test error falls in **no** region — no proposal beats the uniform baseline ($\alpha = 1$).

- A non-conclusive result: no proposal beat the uniform baseline *here* — but the GRU already sits near the Bayes floor, leaving little for any proposal to recover.

Conclusion

Conclusion

Goal: improve training for neural Bayes estimators by more efficient sampling of the Bayes risk.

- **Rao–Blackwellisation** (needs a conjugate block): a *provably* lower-variance training gradient, for any model or architecture — and can be applied wherever Gibbs sampling is used.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Future work: many more variance-reduction techniques remain to be tried — particularly **control variates**.

Thank you